
ΒΕΛΤΙΣΤΟΠΟΙΗΣΗ

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ ΓΙΑ
ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2025-2026

ΟΜΑΔΑ 5380

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΙΑΝΟΥΑΡΙΟΣ 2026

1 ΕΙΣΑΓΩΓΗ

Η συγκεκριμένη αναφορά συγγράφεται στα πλαίσια του μαθήματος Βελτιστοποίησης από τον φοιτητή Γιάννη Φίλλη (ΑΜ:5380). Σκοπός της εργασίας είναι να βελτιστοποιήσουμε τις παραμέτρους ενός μη γραμμικού μοντέλου ισχύος ενός ραντάρ για εναέριες πτήσεις, χρησιμοποιώντας συνθετικά δεδομένα.

Στα πλαίσια αυτά λοιπόν, υλοποίησα πέντε αλγόριθμους τους οποίους διδαχθήκαμε κατά την διάρκεια του εξαμήνου, τον *Newton Dogleg*, τον *BFGS with Wolfe Conditions line search (BFGS-W)*, τον *Nelder Mead simplex*, τον *Genetic Algorithm* και τον *Particle Swarm Optimization (PSO)*.

Παρακάτω, προσπαθώ να αξιολογήσω την ποιότητα των λύσεων αλλά και την υπολογιστική απόδοσή τους και να τους συγκρίνω. Ως αντικειμενική συνάρτηση χρησιμοποίησα όπως αναφέρει ρητά και στην εκφώνηση, το μέσο τετραγωνικό σφάλμα (*MSE*). Τα μοντέλα εκπαιδεύονται πάνω στο training set (radar_train.txt) και αξιολογούνται πάνω στο test set (radar_test.txt).

Έτσι λοιπόν, δημιούργησα δύο αρχεία, το *algo.py* και το *analysis.py*. Το πρώτο, είναι ο πυρήνας της εργασίας, δηλαδή εκεί είναι όλοι οι αλγόριθμοι οι οποίοι εκπαιδεύονται. Όταν τρέξουν όλοι οι αλγόριθμοι, δημιουργείται ένα αρχείο txt για κάθε αλγόριθμο που περιέχει τα αποτελέσματα (όπως ζητούνται) από κάθε run του κάθε αλγόριθμου. Έπειτα, τρέχοντας το άλλο python script, δημιουργούνται τα csv αρχεία *comparison_matrix.csv*, *correlations.csv* και *descriptive_statistics.csv* τα οποία περιέχουν τα στατιστικά και τις συγκρίσεις των αλγορίθμων όπως αυτά ζητούνται (θα αναλυθούν περισσότερο παρακάτω). Τέλος δημιουργείται και μία εικόνα η οποία περιέχει τα boxplots για κάθε αλγόριθμο για το *MSE_{train}*, *MSE_{test}* και *last_hit* για να μπορούμε να τους συγκρίνουμε και οπτικά. Ως budget όρισα αυτό που ανέφερε στην εκφώνηση, καθώς θεώρησα ότι ήταν το καλύτερο.

2 ALGOS.PY

Πριν προχωρήσω στην ανάλυση των αλγορίθμων, θα εξηγήσω κάποια σημεία του κώδικα προκειμένου να είναι περισσότερο κατανοητός και να εξαλείψω πιθανόν ασάφειες της υλοποίησης.

2.1 BUDGET TRACKER

Όπως ζητείται πρέπει να υπολογίζουμε το κόστος των αλγορίθμων. Για να το κάνω δημιούργησα την κλάση **BudgetTracker**. Αυτή, κρατάει σημαντικές πληροφορίες για τον υπολογισμό του κόστους. Η πρώτη μεταβλητή που έχει είναι η **max_cost** η οποία είναι το μέγιστο κόστος που θέτουμε εξαρχής για τους αλγόριθμους μας. Η επόμενη είναι η **cost** η οποία απλά κρατάει κάθε φορά το κόστος. Έπειτα έχω την **stop_flag** την οποία την δημιούργησα προκειμένου να μπορώ εύκολα να σηματοδοτώ ότι πρέπει να τελειώσει ο αλγόριθμος. Τέλος, έχω την **best_mse** η οποία κρατάει το καλύτερο MSE που έχουμε βρει κάθε δεδομένη στιγμή και το **last_hit** που δείχνει πιο είναι το κόστος στο οποίο βρήκαμε το χαμηλότερο σφάλμα, δηλαδή την βέλτιστη λύση.

Έχω δημιουργήσει και τρεις συναρτήσεις, πέρα από την **_init_** για την διαχείριση του αντικείμενου. Η πρώτη, είναι η **reset()**, η οποία χρησιμοποιείται για να κάνω reset το αντικείμενο (μιας και χρησιμοποιώ το ίδιο για όλους τους αλγόριθμους). Η επόμενη συνάρτηση είναι η **check()** η οποία παίρνει ως όρισμα το κόστος που θα προσθέσουμε στον αλγόριθμο και ελέγχει αν μπορεί να το κάνει (αν έχει διαθέσιμο budget). Αν δεν μπορεί θέτει την μεταβλητή **stop_flag** ως True και επιστρέφει false ενώ αν μπορεί, επιστρέφει true. Τέλος, έχω δημιουργήσει και την **update()** η οποία χρησιμοποιείται για την καταγραφή του πότε είχαμε το καλύτερο mse και για την αύξηση του κόστους.

2.2 ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ

Επίσης έχω δημιουργήσει και κάποιες βοηθητικές συναρτήσεις. Η πρώτη είναι η **get_beta()** την οποία δημιούργησα προκειμένου κάθε φορά, όταν χρειάζεται, να επιστρέφω από τον μετασχηματισμό στον χώρο $[0,1]$, καθώς χρειάζεται για τους υπολογισμούς. Έπειτα έχω την **model_predict()** για την υλοποίηση του παραμετρικού μοντέλου. Να σημειώσω εδώ πως χρησιμοποίησα την **clips** με όρια -100 έως 100 για τον εκθέτη ώστε να είμαι σίγουρος πως η τιμή δεν θα ξεφύγει σε πολύ μεγάλο νούμερο που θα μπορούσε να χαλάσει το πρόγραμμα.

Η επόμενη συνάρτηση είναι η **func()**. Σε αυτήν αρχικά ελέγχουμε αν μπορούμε να κάνουμε τον υπολογισμό (από θέμα κόστους) και αν δεν μπορούμε τότε επιστρέφουμε ένα πολύ μεγάλο νούμερο ως mse προκειμένου να καταλάβει ο αλγόριθμος ότι ένας τέτοιος υπολογισμός δεν γίνεται. Έπειτα ελέγχουμε για τα όρια του w αν τηρούνται. Σε περίπτωση που κάτι τέτοιο δεν ισχύει τότε πάλι επιστρέφει ένα μεγάλο νούμερο ώστε ουσιαστικά να πει στον αλγόριθμο ότι η κατεύθυνση αυτή δεν είναι η σωστή. Αποφάσισα να αυξάνω και το κόστος κατά ένα σε αυτή την περίπτωση γιατί ουσιαστικά ο αλγόριθμος σπαταλάει μία ευκαιρία από το συνολικό budget για την απόπειρα να εξερευνήσει μία περιοχή εκτός ορίων. Έπειτα κάνουμε την πρόβλεψη και υπολογίζουμε το μέσο τετραγωνικό σφάλμα και ενημερώνουμε τον budget tracker για την καινούργια τιμή. Τέλος επιστρέφουμε το mse.

Μία ακόμα συνάρτηση που υλοποίησα είναι η **gradient()**. Αυτή χρησιμοποιείται για τον υπολογισμό των παραγώγων. Με την ίδια λογική όπως και πριν, κάνω έλεγχο για το αν μπορώ να κάνω τον υπολογισμό και παρακάτω έχω και τους ελέγχους για τα όρια. Έπειτα έχω τον υπολογισμό της παραγώγου όπου το υπολόγισα αναλυτικά και δημιούργησα μία μεταβλητή για κάθε όρο της παραγώγου. Στο τέλος κάνουμε update το κόστος και επιστρέφουμε την παράγωγο. Επειδή όμως η grad_beta είναι ως προς τα φυσικά νούμερα (όχι τα κανονικοποιημένα) πολλαπλασιάζω με το DIFF ώστε από τον κανόνα της αλυσίδας να πάρει ο αλγόριθμος το νούμερο που αντιστοιχεί στις κανονικοποιημένες μεταβλητές.

Η άλλη βοηθητική συνάρτηση που υλοποίησα είναι η **hessian()** η οποία υπολογίζει τον Εσσιανό πίνακα. Κάνω τον έλεγχο για το αν μπορώ να κάνω τον υπολογισμό και αν δεν γίνεται απλά επιστρέφει τον πίνακα I. Αυξάνουμε το κόστος κατά 15 μονάδες όπως λέει η εκφώνηση και έχω δημιουργήσει μία συνάρτηση **raw_grad()** η οποία είναι η ίδια με την gradient() απλά χωρίς τον tracker διότι διαφορετικά θα έβγαине το κόστος 25 και όχι 15. Παρακάτω υπολογίζουμε τον Εσσιανό με την βοήθεια της συνάρτησης αυτής. Στο τέλος επιστρέφω τον μέσο όρο του Εσσιανού με του αναστροφού του ώστε έτσι να καταφέρω να μειώσω το error που μπορεί να προκύπτει από τους υπολογισμούς και να είναι όσο περισσότερο γίνεται συμμετρικός.

2.3 PARAMETERS TUNING

Για τους αλγορίθμους PSO και GA δημιούργησα την συνάρτηση **run_tuning()** η οποία αυτό που κάνει είναι να βρει τις καλύτερες παραμέτρους για τους πληθυσμούς και για το mutation rate για τον γενετικό αλγόριθμο. Έθεσα εγώ διάφορες τιμές και για κάθε τιμή έτρεξα τον κάθε αλγόριθμο με κόστος 10000. Κάθε φορά το τρέχω 30 φορές και παίρνω τον μέσο όρο του mse και υπολογίζω τις καλύτερες παραμέτρους. Επίσης τρέχουν με τα ίδια αρχικά σημεία ώστε τυχόν διαφορές να οφείλονται μόνο στις παραμέτρους και όχι πχ σε καλύτερα αρχικά σημεία.

Για την GA οι καλύτεροι παράμετροι ήταν pop=70 και mut=0.15 και για την PSO pop=80.

2.4 ΑΛΓΟΡΙΘΜΟΙ

Στην BFGS με wolfe conditions, υλοποίησα τις strong wolf conditions, με βάση τις διαφάνειες του μαθήματος. Επίσης σε πολλά σημεία για σιγουριά ότι οι παράμετροι δεν θα φεύγουν εκτός των ορίων [0,1] χρησιμοποιώ την clip ώστε να το ρυθμίζω. Επίσης, στον γενετικό αλγόριθμο επέλεξα να χρησιμοποιήσω την τεχνική tournament selection.

2.5 ΑΝΑΠΑΡΑΓΩΓΗ ΠΕΙΡΑΜΑΤΩΝ

Για την αναπαραγωγή των πειραμάτων χρησιμοποίησα ως seed τον αριθμό 3 και επίσης έχω αποθηκεύσει τα αρχικά σημεία που δημιούργησα στο αρχείο initial_points.txt. Ο κώδικας κάνει έλεγχο για το αν υπάρχει το αρχείο και το διαβάζει, διαφορετικά δημιουργεί καινούργια.

3 ΑΝΑΛΥΣΗ

Εφόσον ολοκληρώσαμε αυτά, μπορούμε πλέον να πάμε παρακάτω και να απαντήσουμε στα ερευνητικά ερωτήματα τις άσκησης, κάνοντας χρήση των στατιστικών και των μετρικών που υπολογίσαμε.

3.1 ΕΡΩΤΗΜΑ 1

Το πρώτο που θέλουμε να απαντήσουμε είναι πόσο καλές είναι οι λύσεις που υπολογίζει κάθε αλγόριθμός μας ως προς το μέσο τετραγωνικό σφάλμα εκπαίδευσης και γενίκευσης. Για να το κάνουμε αυτό θα χρησιμοποιήσουμε τα αποτελέσματα που είναι καταγεγραμμένα στο αρχείο *descriptive_statistics.csv*.

Algorithm	Metric	Mean	Median	Std Dev.	Min	Max
NewtonDG	MSE_{train}	1.3601940000000001	1.059747	1.6534955367425035	1.022046	10.113208
	MSE_{test}	1.1267546333333334	0.873297	1.4017070817261132	0.850213	8.548067
	LastHit	297.5	189.0	218.1048071376505	105.0	924.0
BFGS-W	MSE_{train}	781.2313624	513.068113	893.187860932061	1.022046	3234.529189
	MSE_{test}	783.1563929333333	510.3008215	895.2586291492505	0.862295	3242.303413
	LastHit	143.46666666666667	79.0	139.64159377722555	36.0	580.0
Nelder-Mead	MSE_{train}	2.2432947333333337	1.464913	2.305555519710267	1.022046	10.438241
	MSE_{test}	1.7797846666666667	0.9289339999999999	1.8697342310813228	0.864174	8.247183
	LastHit	1096.4333333333334	1018.5	443.80623943441753	295.0	2032.0
Genetic Alg	MSE_{train}	1.0477687999999998	1.03857	0.02912279955091471	1.022462	1.127923
	MSE_{test}	0.8730339333333333	0.8704289999999999	0.012690248194451586	0.854039	0.906359
	LastHit	94806.03333333334	97881.5	5595.187448607729	79569.0	99988.0
PSO	MSE_{train}	1.4948460333333335	1.4661089999999999	0.41452167117347094	1.022047	2.374152
	MSE_{test}	1.1518484	0.9364595	0.4096678614565538	0.863113	1.84994
	LastHit	85987.9	99345.0	23515.818713118962	23454.0	99963.0

Θα αναλύσουμε ξεχωριστά τον κάθε αλγόριθμο. Αρχικά, για τον **Newton Dogleg** μπορούμε να δούμε πως έχει την δεύτερη χαμηλότερη μέση τιμή για το MSE_{test} γεγονός που υποδεικνύει πως έχει την δεύτερη καλύτερη γενικευτική ικανότητα. Επίσης έχει και το δεύτερο χαμηλότερο MSE_{train} το οποίο δείχνει ότι και κατά την εκπαίδευση τα πηγαίνει πολύ καλύτερα από τους υπόλοιπους. Ωστόσο μπορούμε και στις δύο μετρικές να δούμε πως η μέγιστες τιμές είναι αρκετά υψηλές και σε συνδυασμό με την τιμή της τυπικής απόκλισης καταλήγουμε στο συμπέρασμα πως είναι λίγο ασταθής και πιθανόν αυτή η αστάθεια που παρατηρείται να οφείλεται στο αρχικό σημείο, όπου σε κάποιες περιπτώσεις θα εγκλωβίζεται σε τοπικά ελάχιστα.

Ο επόμενος αλγόριθμος είναι ο **BFGS-W** ο οποίος είναι ξεκάθαρο ότι αποτυγχάνει. Έχει με πολύ μεγάλη διαφορά τα μεγαλύτερα MSE_{test} και MSE_{train} το οποίο δείχνει ότι ούτε κατά την εκπαίδευση γίνεται καλή δουλειά αλλά και προφανώς δείχνει αδυναμία στην γενίκευση. Παγιδεύεται μακριά από την βέλτιστη περιοχή. Ίσως αυτό να οφείλεται στο πως έχει υλοποιηθεί όπου τον περιορίζουμε στο $[0,1]$ οπότε παγιδεύεται σε κάποιο τοπικό ελάχιστο. Επίσης ίσως να είναι και πολύ αυστηρές οι συνθήκες wolfe καθώς όπως φαίνεται από την μετρική LastHit κάνει αρκετά λίγους συναρτησιακούς υπολογισμούς. Ωστόσο θα πρέπει να επισημάνουμε πως και αυτός καταφέρνει να έχει ικανοποιητικές τιμές σε αυτές τις μετρικές όπως φαίνεται στην στήλη του ελάχιστου όπου πετυχαίνει σχεδόν ίδια τιμή με τον Newton Dogleg.

Επόμενος αλγόριθμος είναι ο **Nelder-Mead** όπως φαίνεται να είναι αξιόπιστος. Αν και το MSE_{test} είναι λίγο υψηλό, φαίνεται πετυχαίνει αρκετά καλή γενίκευση. Ωστόσο βλέποντας την σχετικά υψηλή τυπική απόκλιση μπορούμε να καταλήξουμε στο συμπέρασμα ότι εξαρτάται ως ένα βαθμό και από τα αρχικά σημεία.

Προχωρώντας στους πληθυσμιακούς αλγορίθμους ο πρώτος που συναντάμε είναι ο Γενετικός αλγόριθμος. Πετυχαίνει εξαιρετικό μέσο MSE_{test} καθώς και πάρα πολύ καλό MSE_{train} λίγο χαμηλότερο από αυτό του Newton Dogleg. Ταυτόχρονα έχει πάρα πολύ χαμηλή τυπική απόκλιση γεγονός που υποδεικνύει ότι δεν επηρεάζεται από το αρχικό σημείο.

Τέλος, έχουμε την **PSO** η οποία έχει πολύ καλή επίδοση και κατά την εκπαίδευση αλλά και πολύ χαμηλό τετραγωνικό σφάλμα γενίκευσης, ωστόσο δεν είναι όσο καλός ο γενετικός αλγόριθμος.

Κοιτώντας όμως και το last hit μπορούμε να δούμε ότι ενώ έχουν πολύ καλό μέσο τετραγωνικό σφάλμα οι πληθυσμιακοί αλγόριθμοι εξαντλούν πάντα το όριο των υπολογισμών (κάτι που θα φανεί και γραφικά παρακάτω από το boxplot) ενώ ο Newton Dogleg είναι πολύ πιο γρήγορος.

3.2 ΕΡΩΤΗΜΑ 2

Αρα με βάση τα παραπάνω αν θέλαμε να τους συγκρίνουμε κοιτώντας το μέσο τετραγωνικό σφάλμα γενίκευσης, μπορούμε να τα κατατάξουμε όπως παρακάτω:

Genetic Algorithm > NewtonDG > Nelder Mead > PSO > BFGS

Ας μελετήσουμε και τον πίνακα που δημιουργείται στο **comparison_matrix.csv** όπου συγκρίνουμε τους αλγορίθμους όπου συγκρίνουμε τις μετρικές τους με την χρήση του Wilcoxon rank-sum και έχοντας ως πρώτη μετρική το μέσο τετραγωνικό σφάλμα γενίκευσης.

	NewtonDG	BFGS	NM	GA	PSO	Total (W/L/T)
NewtonDG	=	+	+	+	+	4/0/1
BFGS-W	-	=	-	-	-	0/4/1
NM	-	+	=	-	+	2/2/1
GA	-	+	+	=	+	3/1/1
PSO	-	+	-	-	=	1/3/1

Όπως μπορούμε να δούμε, ξεκάθαρα ο Newton Dogleg είναι πρώτος. Παρόλο που έχει λίγο υψηλότερο τετραγωνικό σφάλμα από τον Γενετικό Αλγόριθμο, είναι ξεκάθαρα καλύτερος σε κατανάλωση υπολογιστικών πόρων. Αμέσως μετά ακολουθεί ο Γενετικός αλγόριθμος, έπειτα ο Nelder Mead και τέλος ο PSO και ο BFGS. Ο PSO παρόλο που φαίνεται να βρίσκει αρκετά καλές τιμές, η πολύ υψηλή τιμή του LastHit, που δείχνει ότι χρησιμοποιεί σχεδόν όλο το budget δεν τον καθιστά και ιδιαίτερα ελκυστικό συγκριτικά με τους άλλους.

3.3 ΕΡΩΤΗΜΑ 3

Το τελευταίο ερευνητικό ερώτημα που καλούμαστε να απαντήσουμε είναι πως σχετίζεται η ποιότητα των λύσεων στην εκπαίδευση με την αντίστοιχη ποιότητα γενίκευσης για κάθε αλγόριθμο. Για την απάντηση σε αυτό το ερώτημα θα

χρησιμοποιήσουμε τα αποτελέσματα από το αρχείο **correlations.csv** όπου πραγματοποιήσαμε ελέγχους με χρήση τριών διαφορετικών συντελεστών, των Pearson, Spearman και Kendall.

Παρακάτω παραθέτω τον πίνακα ο οποίος περιέχει τα αποτελέσματα των ελέγχων που κάναμε.

Algorithm	Pearson	Spearman	Kendall
NewtonDG	0.9998	0.1732	0.1414
BFGS-W	1.0	0.9884	0.9529
NM	0.9965	0.9661	0.9126
GA	-0.1954	-0.2525	-0.0713
PSO	0.8785	0.9348	0.7831

Η τεχνική Pearson ελέγχει την γραμμική συσχέτιση μεταξύ των δύο μεταβλητών (στην περίπτωση μας του MSE_{test} και MSE_{train}). Ένα αρνητικό που έχει η συγκεκριμένη τεχνική είναι ότι είναι αρκετά ευαίσθητη σε ακραίες τιμές, δηλαδή αν σε ένα πείραμα ξεφύγουν οι τιμές των μετρικών θα επηρεαστεί σημαντικά η τιμή του Pearson.

Η τεχνική Spearman από την άλλη, μετρά την μονοτονική σχέση που μπορεί να έχουν τα δεδομένα μας. Αυτό που κάνει είναι να ελέγχει την κατάταξη των τιμών και ακριβώς τις ίδιες τις τιμές. Επίσης είναι περισσότερο ανθεκτικός στις ακραίες τιμές.

Τέλος, η τεχνική Kendall μετράει την συσχέτιση κατάταξης βασιζόμενος σε ζεύγη και ουσιαστικά ελέγχει την πιθανότητα η σειρά κατάταξης δύο μετρήσεων να είναι ίδια.

Οπότε τώρα θα πάμε να αναλύσουμε ξεχωριστά τα αποτελέσματα για κάθε αλγόριθμο. Ξεκινώντας από τον Newton Dogleg, δίνει πολύ υψηλή τιμή Pearson ωστόσο η Spearman και η Kendall είναι αρκετά χαμηλές. Αυτή η μεγάλη διαφορά πιθανόν να οφείλεται σε κάποιες πολύ ακραίες τιμές και η χαμηλή τιμή του Spearman μας δείχνει πως η γενίκευση δεν είναι επαρκής.

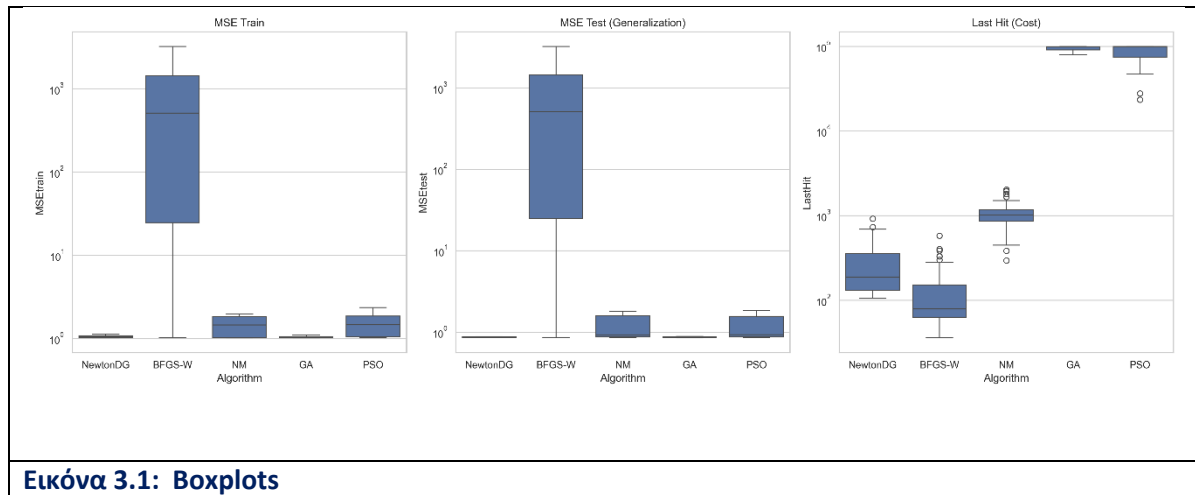
Η επόμενη είναι η BFGS. Αυτή δείχνει πολύ υψηλή συσχέτιση, ωστόσο δεν είναι καλό, απλά σημαίνει (με βάση και τα αποτελέσματα που πήραμε) ότι σταθερά αποτυγχάνει.

Αν κοιτάξουμε τώρα τον Nelder-Mead και τον PSO βλέπουμε πως οι μετρικές έχουν πολύ υψηλές τιμές, γεγονός που υποδεικνύει πολύ ισχυρή και συνεπή συσχέτιση. Μπορούμε εδώ να συμπεράνουμε πως όσο καλύτερα εκπαιδεύονται τόσο καλύτερα γενικεύουν.

Τέλος, ο γενετικός αλγόριθμος φαίνεται να έχει διαφορετική συμπεριφορά. Ποιο συγκεκριμένα, έχει αρνητικές τιμές και στις τρεις μετρικές. Όπως αναφέρει και στην εκφώνηση, αυτό ίσως να οφείλεται και σε κάποιο overfitting που μπορεί να έγινε.

3.4 BOXPLOT

Τέλος, θα αναλύσουμε και το boxplot που δημιουργήσαμε.



Εικόνα 3.1: Boxplots

Ξεκινώντας λοιπόν από το MSE_{train} μπορούμε να δούμε πως ξεχωρίζει ο BFGS ο οποίος έχει πολύ υψηλές τιμές. Ακολουθεί ο Nelder Mead όπου και η διάμεσος φαίνεται να είναι λίγο υψηλή όπως και του PSO. Οι Newton Dogleg και Genetic Algorithm φαίνεται να τα πηγαίνουν πολύ καλύτερα.

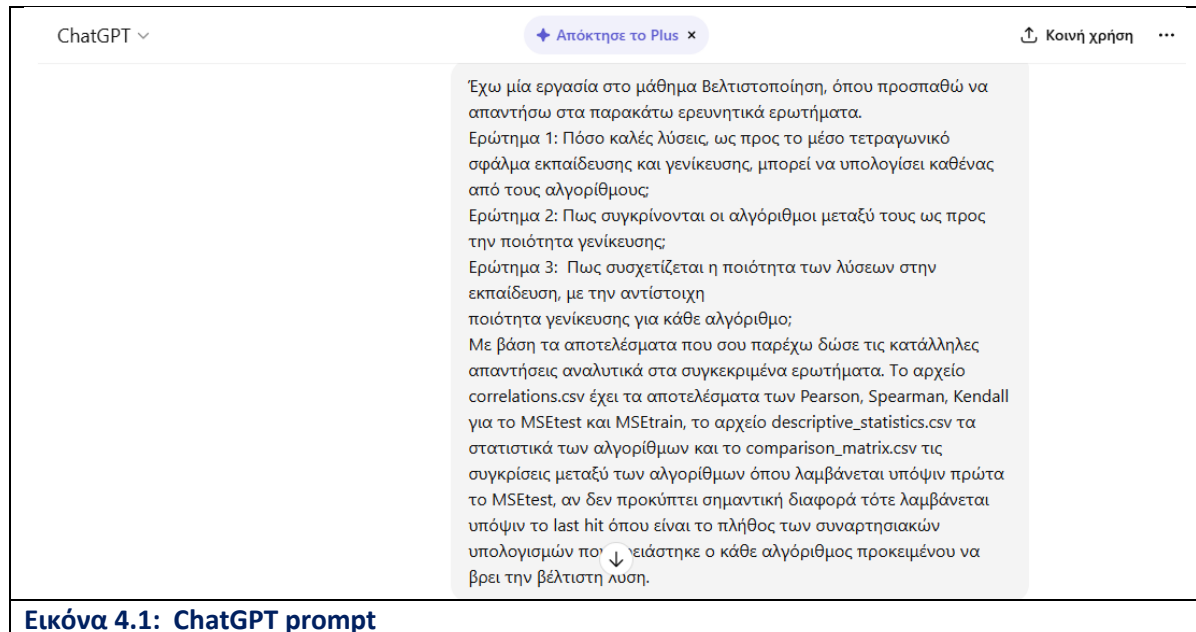
Προχωρώντας στο MSE_{test} ο BFGS φαίνεται να έχει την ίδια συμπεριφορά. Ο Nelder Mead φαίνεται να έχει το ίδιο «κουτάκι» γεγονός που υποδεικνύει πως το 50% των πειραμάτων έχει περίπου παρόμοιες τιμές με την εκπαίδευση, ωστόσο η διάμεσος είναι πολύ χαμηλά. Οι υπόλοιποι τρεις αλγόριθμοι φαίνεται πως έχουν χαμηλότερο τετραγωνικό σφάλμα γενίκευσης από ότι εκπαίδευσης.

Η τρίτη και τελευταία μετρική που εξετάζουμε είναι η Last Hit, όπου αντιπροσωπεύει τους απαιτούμενους συναρτησιακούς υπολογισμούς για να εντοπίσει την βέλτιστη λύση. Αυτό που μπορούμε να παρατηρήσουμε είναι πως ο γενετικός αλγόριθμος καθώς και η PSO κοστίζουν πολύ (κάτι που παρατηρήθηκε και κατά την εκτέλεση του προγράμματος), καθώς χρειάζονται και τους περισσότερους συναρτησιακούς υπολογισμούς. Έπειτα ακολουθεί ο Nelder Mead και τέλος αρκετά κοντά μεταξύ τους φαίνεται να είναι ο Newton Dogleg και ο BFGS.

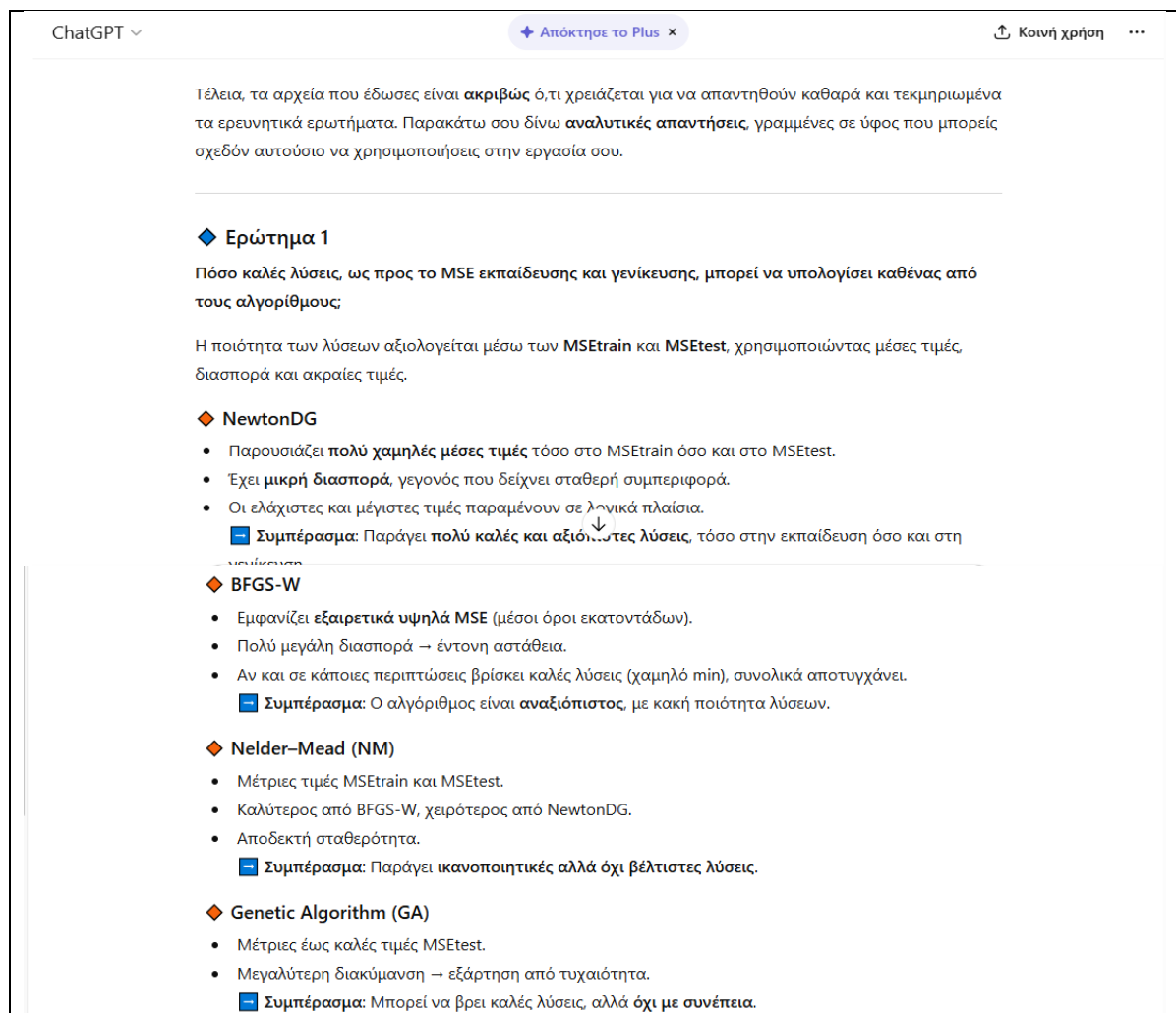
4 MONTELO AI

Στο τελευταίο σκέλος της συγκεκριμένης εργασίας αυτό που θα κάνω είναι να δώσω τα αποτελέσματα στο **ChatGPT** και θα του ζητήσω να μου κάνει και αυτό την δικιά του ανάλυση των αποτελεσμάτων των δεδομένων.

Σαν input του έδωσα τα τρία αρχεία csv και το παρακάτω prompt.



Μετά από σκέψη, μου έδωσε τις παρακάτω απαντήσεις.



◆ Particle Swarm Optimization (PSO)

- Καλές τιμές MSEtest, κοντά στον NM.
 - Καλύτερη σταθερότητα από GA.
- ➡ Συμπέρασμα: Αξιόπιστος στοχαστικός αλγόριθμος, με καλή ποιότητα λύσεων.

**Εικόνα 4.2: Ερώτημα 1**

ChatGPT ▾ [+ Απόκτησε το Plus](#) x [↑ Κοινή χρήση](#) ...

◆ Ερώτημα 2

Πώς συγκρίνονται οι αλγόριθμοι μεταξύ τους ως προς την ποιότητα γενίκευσης;

Η σύγκριση βασίζεται:

1. Πρώτα στο MSEtest
2. Αν δεν υπάρχει στατιστικά σημαντική διαφορά → στο LastHit

Σύγκριση (comparison matrix)

- **NewtonDG**: νικά όλους τους άλλους (4 νίκες, 0 ήττες)
- **GA**: 3 νίκες, 1 ήττα
- **NM**: 2 νίκες, 2 ήττες
- **PSO**: 1 νίκη, 3 ήττες
- **BFGS-W**: 0 νίκες, 4 ήττες

Τελική κατάταξη γενίκευσης:

1. NewtonDG
2. GA
3. NM
4. PSO
5. BFGS-W

➡ Συμπέρασμα:

Ο NewtonDG υπερέχει ξεκάθαρα στη γενίκευση. Οι στοχαστικοί αλγόριθμοι (GA, PSO) υπερτερούν του BFGS-W, ενώ ο NM βρίσκεται ενδιάμεσα.

Εικόνα 4.3: Ερώτηση 2

◆ Ερώτημα 3

Πώς συσχετίζεται η ποιότητα εκπαίδευσης με την ποιότητα γενίκευσης για κάθε αλγόριθμο;

Η συσχέτιση μελετάται μέσω **Pearson**, **Spearman** και **Kendall** μεταξύ MSEtrain και MSEtest.

◆ **NewtonDG**

- Pearson ≈ 1
- Spearman / Kendall πολύ χαμηλές
 - ☐ Υπάρχει **γραμμική συσχέτιση**, αλλά όχι μονοτονική.
 - ☐ Δείχνει ότι μικρό MSEtrain δεν εγγυάται πάντα αντίστοιχα μικρό MSEtest.

◆ **BFGS-W**

- Και οι τρεις συντελεστές ≈ 1
 - ☐ **Ισχυρότατη συσχέτιση**: όταν αποτυγχάνει στην εκπαίδευση, αποτυγχάνει και στη γενίκευση.
 - ☐ Δεν εμφανίζει overfitting — απλώς κακή σύγκλιση.

◆ **NM**

- Υψηλές τιμές σε όλους τους δείκτες
 - ☐ **Καλή και συνεπής σχέση** εκπαίδευσης-γενίκευσης.

◆ **GA**

- Αρνητική συσχέτιση
 - ☐ **Ασθενής ή ανύπαρκτη σχέση**.
 - ☐ Καλό MSEtrain δεν συνεπάγεται καλό MSEtest → έντονη στοχαστικότητα.

◆ **PSO**

- Υψηλές Spearman & Kendall
 - ☐ Καλή μονοτονική σχέση.
 - ☐ Βελτίωση στην εκπαίδευση συνήθως οδηγεί και σε καλύτερη γενίκευση.

✅ **Συνολικό συμπέρασμα**

- **NewtonDG**: ο καλύτερος αλγόριθμος συνολικά.
- **GA & PSO**: καλή γενίκευση αλλά λιγότερη σταθερότητα.
- **NM**: ισορροπημένη λύση.
- **BFGS-W**: αποτυγχάνει τόσο σε εκπαίδευση όσο και γενίκευση.

Αν θέλεις, στο επόμενο μήνυμα μπορώ:

- 📄 να σου τα μετατρέψω σε **ακαδημαϊκό ύφος LaTeX**
- 📊 να προτείνω **γραφήματα** που "δένουν" τέλεια με τα αποτελέσματα
- 🗣️ ή να σου γράψω **discussion / threats to validity** για άριστο βαθμό

📄 📊 🗣️ ⬆️ ↻️ ...

Εικόνα 4.4: Ερώτηση 3

Επειδή είχε περιορισμό στο πόσα αρχεία μπορούσα να στείλω ταυτόχρονα, τα boxplots τα έστειλα σε διαφορετική ερώτηση παρακάτω.

Με βάση και τα παρακάτω boxplots, τι θα μπορούσες να σχολιάσεις πάνω στα συγκεκριμένα ερωτήματα?

Εικόνα 4.5: ChatGPT prompt 2

ChatGPT

Απόκτήσε το Plus

Κοινή χρήση

Τέλεια, τα boxplots που δείχνεις **κουμπώνουν πολύ καθαρά** με τα ποσοτικά αποτελέσματα των CSV και μας επιτρέπουν να δώσουμε **ποιοτική ερμηνεία** στα ερευνητικά ερωτήματα. Θα τα σχολιάσω **στοχευμένα ανά ερώτημα**, όπως θα τα έγραφες σε εργασία Βελτιστοποίησης.

◆ **Ερώτημα 1**

Πόσο καλές λύσεις μπορεί να υπολογίσει κάθε αλγόριθμος (MSEtrain & MSEtest);

Από τα boxplots MSEtrain & MSEtest

GA

- Εμφανίζει **πολύ χαμηλές τιμές MSE** τόσο στην εκπαίδευση όσο και στη γενίκευση.
- Τα boxplots είναι **εξαιρετικά "σφιχτά"** → πολύ μικρή διασπορά.
- Αυτό δείχνει **σταθερή σύγκλιση σε καλές λύσεις**, ανεξάρτητα από την εκτέλεση.

NewtonDG

- Πολύ χαμηλό MSEtrain και MSEtest, συγκρίσιμο με τον GA.
- Ελάχιστη διασπορά → **αξιόπιστος και προβλέψιμος**.
- Ελαφρώς χειρότερος από GA στο test, αλλά όχι δραματικά.

PSO

- Καλές αλλά **σαφώς χειρότερες λύσεις** από GA / NewtonDG.
- Η διασπορά είναι μικρή, άρα ο αλγόριθμος είναι σταθερός, αλλά **συγκλίνει σε υποδεέστερα τοπικά άκρα**.

NM

- Υψηλότερα MSE και μεγαλύτερη διασπορά.
- Υπάρχουν εκτελέσεις που αποδίδουν καλά, αλλά και πολλές που όχι → **αστάθεια**.

BFGS-W

- Boxplots σε **λογαριθμική κλίμακα (!)**
- Πολύ μεγάλες τιμές και τεράστια διασπορά.
- Ξεκάθαρη αποτυχία να βρει καλές λύσεις.

✅ **Συμπέρασμα Ερωτήματος 1**

Τα boxplots δείχνουν καθαρά ότι **GA και NewtonDG** παράγουν τις καλύτερες και πιο σταθερές λύσεις, ενώ ο **BFGS-W** **αποτυγχάνει πλήρως** στο συγκεκριμένο πρόβλημα.

↓

Εικόνα 4.6: Ερώτηση 1

◆ Ερώτημα 2

Πώς συγκρίνονται οι αλγόριθμοι ως προς την ποιότητα γενίκευσης;

Από το boxplot MSEtest

- **GA** έχει:
 - το χαμηλότερο median MSEtest
 - σχεδόν μηδενική διασπορά
→ κορυφαία γενίκευση
- **NewtonDG**:
 - πολύ κοντινή επίδοση
 - επίσης εξαιρετική σταθερότητα
→ ελαφρώς χειρότερος από GA, αλλά ανταγωνιστικός
- **PSO & NM**:
 - εμφανώς υψηλότερα MSEtest
 - μεγαλύτερη μεταβλητότητα
→ κατώτερη γενίκευση
- **BFGS-W**:
 - ακραίες τιμές → μη αποδεκτή γενίκευση

**Σε συνδυασμό με το boxplot Last Hit**

- **GA & PSO**: τεράστιο υπολογιστικό κόστος ($\sim 10^5$)
- **NewtonDG**: πολύ χαμηλό κόστος ($\sim 10^2$)
- Άρα:
 - GA → καλύτερη γενίκευση
 - NewtonDG → καλύτερος συμβιβασμός ποιότητας-κόστους

✓ Συμπέρασμα Ερωτήματος 2

Αν εξεταστεί μόνο η ακρίβεια, ο **GA** υπερέχει.

Αν ληφθεί υπόψη και το κόστος, ο **NewtonDG** εμφανίζεται συνολικά ανώτερος.

**Εικόνα 4.7: Ερώτηση 2****◆ Ερώτημα 3**

Πώς συσχετίζεται η ποιότητα εκπαίδευσης με τη γενίκευση;

Τα boxplots βοηθούν εδώ οπτικά:

Παρατηρήσεις**GA**

- Παρόμοια κατανομή MSEtrain και MSEtest.
- Πολύ μικρή απόσταση μεταξύ τους.
- Υποδηλώνει καλή γενίκευση χωρίς overfitting.

NewtonDG

- Σχεδόν ταυτόσημα boxplots train/test.
- Δείχνει ισχυρή σχέση εκπαίδευσης-γενίκευσης.
- Ωστόσο, η πολύ "σφιχτή" σύγκλιση υποδηλώνει εξάρτηση από το τοπικό ελάχιστο.

PSO & NM

- Μεγαλύτερο "άνοιγμα" μεταξύ train και test.
- Όταν η εκπαίδευση δεν είναι καλή, η γενίκευση υποβαθμίζεται έντονα.



BFGS-W

- Κακή εκπαίδευση → κακή γενίκευση (οπτικά και απόλυτα).

Σύνδεση με correlations

- Η οπτική εικόνα επιβεβαιώνει:
 - υψηλές συσχετίσεις για τοπικές μεθόδους
 - χαμηλότερη (αλλά υγιή) συσχέτιση για GA → ανθεκτικότητα

✓ Συμπέρασμα Ερωτήματος 3

Οι τοπικοί αλγόριθμοι παρουσιάζουν ισχυρή εξάρτηση μεταξύ εκπαίδευσης και γενίκευσης, ενώ ο GA επιτυγχάνει καλή γενίκευση ανεξάρτητα από μικρές διακυμάνσεις στο training, γεγονός που τον καθιστά πιο ανθεκτικό.

Εικόνα 4.8: Ερώτηση 3

5 ΣΧΟΛΙΑΣΜΟΣ ΑΠΟΤΕΛΕΣΜΑΤΩΝ CHATGPT

Το μοντέλο AI φαίνεται να συμφωνεί με τα δικά μου αποτελέσματα. Αναδεικνύει τον Newton Dogleg ως τον καλύτερο αλγόριθμο συνολικά ενώ παράλληλα σχολιάζει και για τον Γενετικό Αλγόριθμο την αποδοτικότητά του. Επίσης δεν φάνηκε να κάνει κάποιο σχόλιο για τις αρνητικές τιμές των μετρικών και κάποιο πιθανό overfitting, αλλά σχολιάζει ότι από το boxplot καταλαβαίνει ότι δεν υπάρχει overfitting στον συγκεκριμένο αλγόριθμο.

Τέλος σχολιάζει και για τον BFGS την πολύ κακή επίδοση που έχει τόσο στο training όσο και στο test.

6 ΣΥΝΟΨΗ

Ολοκληρώνοντας αυτήν την εργασία, μελέτησα σε μεγαλύτερο βάθος τους αλγορίθμους που διδαχθήκαμε και τους εφάρμοσα σε ένα αρκετά ρεαλιστικό πρόβλημα. Όπως είδαμε, ο Newton Dogleg και ο Γενετικός Αλγόριθμος είχαν φοβερές επιδόσεις, με τον Nelder Mead και την PSO να ακολουθούν. Ο BFGS από την άλλη είχε πολύ απογοητευτική επίδοση και θα είχε ενδιαφέρον να εφαρμοστεί είτε με διαφορετικές παραμέτρους είτε χωρίς να εφαρμοστούν οι strong wolfe conditions.